

EOAT Command Center Feature Expansion Specification

Purpose

This document defines the next major improvement plan for the EOAT Command Center app. The goal is to turn the app from a strong audit/reporting dashboard into a guided engineering workflow system that helps the user know what matters, what is missing, what is suspicious, what can be repaired automatically, and what should become a follow-up action.

The central idea is:

Build an Audit Coach + Open Items system that converts validation warnings, hidden-field logic, annotations, notes, missing evidence, scheduled report status, and audit completion state into one guided workflow.

The app should not merely display data. It should actively guide the audit process, reduce repeated manual checks, prevent stale workbook data, and make daily project execution faster and more reliable.

This document covers all proposed improvement areas except the previously discussed full Robot Info entity system. Robot Info should remain deliberately small for now because the Master Press List already contains the deeper robot details.

Scope

Included

This plan includes:

1. Audit Coach mode.
2. Annotation suggestion workflow.
3. Unified Open Items / Action Board.
4. Finish This Audit workflow.
5. Dirty-form protection and draft recovery.
6. Audit page decomposition.
7. Config-driven defaults.
8. Gripper presets as a small configurable parts reference.
9. Structured validation findings.
10. Hidden-field reason visibility.
11. Global command palette.
12. Global search.

13. Dashboard cache hardening.
14. Structured performance logging.
15. Scheduled report calendar preview.
16. Catch-up summaries.
17. Smarter scheduled report content.
18. Task Scheduler preflight diagnostics.
19. Field history and audit revision history.
20. Backup retention cleanup.
21. Optional local SQLite mirror for audit state.
22. SQLite full-text search for notes/tags.
23. Photo evidence coverage.
24. Phone-friendly photo intake support.
25. Pilot candidate evidence packets.
26. FMEA generation from tags/issues.
27. Standards compliance scoring.
28. Compatibility impact preview.
29. Consistency checks between physical and compatible rows.
30. Press View / Machine View.
31. Workbook lock detection.
32. Validation severity levels.
33. One-click safe fixes.
34. App-level event bus.
35. Page registry.
36. Page lifecycle hooks.
37. Expanded Settings page.
38. Release readiness page.
39. Staged-file NDA scanner.

Explicitly Excluded

The previously proposed full Robot Info entity system is excluded for now.

Robot Info should remain a small workbook for only these robot-side pneumatic details:

- Robot Vacuum Circuits.
- Robot Pressure Circuits.
- Robot Interchangeable Circuits.

The Master Press List remains the source for deeper robot details such as robot type, model/controller, serial information, and press/robot reference metadata. The app should not duplicate that full robot reference system inside Robot_Info.xlsx.

Important Constraint

The app must remain local-first and NDA-safe.

Real project data must stay outside the GitHub repository. The repository should continue to contain only source code, generic documentation, sanitized templates, tests, and synthetic demo data.

Desired Final Result

At the end of this improvement pass, EOAT Command Center should feel like a guided engineering assistant rather than a collection of disconnected pages.

The user should be able to open the app and immediately understand:

- Which audits are incomplete.
- Which fields are missing and actually matter.
- Which hidden fields were intentionally marked N/A.
- Which fields contain stale values that should not exist.
- Which notes/tags/follow-ups are still open.
- Which scheduled summaries ran, failed, or were missed.
- Which reports are ready.
- Which pilot candidates have strong evidence.
- Which validation findings are safe to repair automatically.
- Which items require a human decision.

The app should make common workflows faster:

- Finish an existing audit.
- Review all open items.
- Apply suggested tags.
- Generate catch-up reports.
- Validate and repair workbook issues.
- Navigate directly to a machine, audit, field, note, tag, or report.
- Prepare weekly and final handoff materials with less manual hunting.

The final product should feel like a small internal engineering operations console: practical, evidence-based, fast, and difficult to accidentally corrupt.

Design Principles

1. Truth Over Completeness Theater

The app should not reward filled cells if those cells are physically meaningless.

Example:

- A Mechanical / Gripper EOAT should not require vacuum cup material.

- A system with Electrical/Wiring Present? = No should not require electrical quick-disconnect details.
- A row marked as compatible should not be treated exactly like a physical audit observation.

The app should distinguish:

- Missing because the user has not checked yet.
- N/A because the field physically does not apply.
- Unknown / Not Checked because the field applies but has not been confirmed.
- Stale because the field should not apply but contains a meaningful value.
- Conflict because values contradict the EOAT type or other fields.

2. Fast First, Deep Second

The dashboard should open quickly with cached information. Expensive workbook scans should run only during deep refresh or background workflows.

The user should never have to wait through a full workbook recalculation just to open the app.

3. Guided Workflow, Not More Buttons

Adding features should not mean adding 47 disconnected buttons. Every feature should help answer at least one of these questions:

- What should I do next?
- Why does this matter?
- Where is the problem?
- Can the app fix it safely?
- Does this require a human decision?

4. Evidence-Based Reporting

Reports should be generated from actual project events, audit changes, notes, tags, validation findings, and open items whenever possible.

Reports should not rely on generic filler text unless no real project data exists.

5. Workbooks Are Deliverables, Not the Only Brain

Excel workbooks remain important final deliverables and human-readable records. However, the app can use SQLite, caches, activity logs, and structured findings internally to improve speed and reliability.

6. Safe by Default

Every file-changing operation should follow safe behavior:

- Preview when practical.

- Back up before workbook changes.
 - Avoid overwrites unless explicitly allowed.
 - Log activity.
 - Report files created/modified.
 - Handle locked workbooks gracefully.
 - Never push real company data to GitHub.
-

Feature Group A: Audit Guidance and Completion

A1. Audit Coach Mode

Goal

Add a persistent guidance panel to the EOAT Audit page that tells the user what still matters for the current audit.

Problem It Solves

The audit form now contains many fields, tabs, conditional visibility rules, annotations, robot-side pneumatic fields, compatibility behavior, and validation warnings. Without a guide, the user can fill out a lot of information and still miss important applicable fields.

The Audit Coach should reduce hunting, guessing, and accidental incomplete saves.

User-Facing Behavior

The Audit Coach panel should appear on the right side of the EOAT Audit page or as a collapsible side panel.

It should show:

- Current audit mode: New, Edit, Duplicate, Compatibility Source, etc.
- Current Audit ID.
- Machine number.
- EOAT type.
- Overall completion state.
- Section-by-section completion.
- Missing applicable fields.
- Fields intentionally hidden as N/A.
- Stale hidden values.
- Hybrid completeness warnings.
- Semantic consistency warnings.
- Missing photo evidence.
- Suggested annotations.
- Next recommended action.

Example display:

```
Audit Coach
-----
Audit: AUD-20260527-004
Machine: 12
EOAT Type: Hybrid
Mode: Editing existing audit

Completion:
Applicable fields complete: 44 / 53
Open important fields: 5
Follow-up accepted fields: 2
Photo evidence missing: 3 categories

Next best action:
Fill Sensor Brand/Model or tag it as Needs Review.

Warnings:
- Hybrid EOAT is missing gripper-side tooling details.
- Photos Taken? is No but audit status is Complete.
- Electrical/Wiring Present? is No, so electrical fields will save as N/A.
```

How It Works

The Audit Coach should use a new core service:

```
core/audit/audit_coach.py
```

Suggested classes:

```
@dataclass
class AuditCoachSectionStatus:
    section_name: str
    applicable_count: int
    completed_count: int
    missing_count: int
    warning_count: int
    fields: list[str]

@dataclass
class AuditCoachFinding:
    severity: str
    category: str
```

```

    field: str | None
    message: str
    reason: str
    recommended_action: str
    action_type: str
    fix_available: bool = False

@dataclass
class AuditCoachSummary:
    audit_id: str
    machine: str
    eoat_type: str
    mode: str
    applicable_field_count: int
    completed_field_count: int
    missing_important_count: int
    section_statuses: list[AuditCoachSectionStatus]
    findings: list[AuditCoachFinding]
    next_action: str

```

Inputs:

- Current form values.
- Field applicability rules.
- Audit section/group definitions.
- Annotation state.
- Validation finding rules.
- Photo evidence state.
- Loaded empty-only fields.
- Robot-side pneumatic small workbook values.

Outputs:

- Display-ready summary.
- Clickable findings.
- Suggested actions.

Implementation Notes

The Audit Coach should not duplicate the validation logic. It should call shared domain functions:

```

core/audit_field_rules.py
core/audit_entries.py
core/validation.py

```

```
core/annotations/service.py
core/photo_evidence.py
```

If new logic is needed, add it in core services, not directly inside the UI.

Expected Final Result

The user can audit faster because the app constantly shows the next useful action instead of forcing the user to manually inspect every tab.

A2. Finish This Audit Workflow

Goal

Create a guided workflow that walks through only the applicable missing fields for a loaded or current audit.

Problem It Solves

The Empty Only view is useful, but the user still has to scan multiple sections and decide what matters. Finish This Audit should turn incomplete audits into a guided checklist.

User-Facing Behavior

Add a button:

```
Finish This Audit
```

When clicked, open a workflow panel or modal:

```
Finish This Audit
-----
Remaining applicable fields: 8
Current field: Sensor Brand/Model
Reason: Sensors Present? is Yes.
Recommended action: Enter model, mark Unknown / Not Checked, or create follow-
up.

[Open Field] [Mark Unknown] [Create Follow-Up] [Tag Needs Review] [Skip] [Next]
```

The workflow should move field-by-field.

Field States

Each field should be classified as:

- Complete.
- Missing.
- Unknown accepted.
- N/A because non-applicable.
- Needs follow-up.
- Conflict.
- Stale hidden value.

Implementation Notes

Create:

```
core/audit/audit_completion.py
```

Suggested functions:

```
def calculate_audit_completion(entry: dict[str, str]) -> AuditCompletionSummary

def next_incomplete_field(entry: dict[str, str], section_order: list[str]) ->
str | None

def mark_field_unknown(entry: dict[str, str], field: str) -> dict[str, str]
```

The UI should call existing navigation methods to jump to fields.

Expected Final Result

Existing audits can be completed methodically without manually switching tabs and guessing what still matters.

A3. Hidden Field Reason Viewer

Goal

Make hidden dynamic form behavior understandable.

Problem It Solves

Dynamic forms can feel broken if fields disappear without explanation. The app should make it clear when a field disappeared because it does not apply.

User-Facing Behavior

Add a section in Audit Coach:

Hidden Fields

Cup Type/Material

Reason: Vacuum tooling fields do not apply to Mechanical / Gripper EOAT.

Electrical Quick Disconnect Type

Reason: Electrical detail fields do not apply when Electrical/Wiring Present? is No.

Add a toggle:

Show Hidden Field Reasons

Optional behavior in Full Audit view:

- Hidden fields remain hidden by default.
- User can temporarily show hidden fields read-only with reason text.

Implementation Notes

Use the existing non-applicable reason logic from field rules. Do not create separate reason strings in the UI.

Expected Final Result

The user trusts the dynamic form because the app explains what it is doing.

A4. Dirty-Form Protection and Draft Recovery

Goal

Prevent accidental loss of unsaved audit work.

Problem It Solves

The audit page can load existing audits, generate new IDs, duplicate audits, run lookups, switch views, tag fields, and save. Losing unsaved work would be painful.

User-Facing Behavior

If the user tries to:

- Load another audit.
- Generate a new Audit ID.
- Duplicate an audit.
- Clear the form.
- Change project root.
- Close the app.

And there are unsaved changes, show:

You have unsaved audit changes.

Options:

[Save Now] [Save Draft] [Discard Changes] [Cancel]

Draft recovery on app start:

Recovered unsaved audit draft from 2026-05-27 18:42.

[Restore Draft] [Discard Draft]

How It Works

Maintain a baseline snapshot of form values when:

- New form is created.
- Existing audit is loaded.
- Audit is saved.
- Form is cleared.

Compare current values to baseline.

Save drafts to ignored local cache:

00_Project_Admin/cache/audit_drafts/

Draft format:

```
{  
  "version": 1,  
}
```

```
"saved_at": "2026-05-27T18:42:00",
"audit_id": "AUD-20260527-004",
"mode": "edit",
"project_root": "...",
"form_values": {},
"baseline_values": {}
}
```

Implementation Notes

Create:

```
core/audit/drafts.py
```

UI integration:

- AuditPage checks dirty state through controller.
- DashboardWindow calls page `can_close()` lifecycle hook before close.

Expected Final Result

The app becomes safer to use in real audit sessions where interruptions happen constantly.

Feature Group B: Open Items, Notes, Tags, and Annotations

B1. Suggested Annotations Workflow

Goal

Turn annotation suggestions from plain text into a reviewable apply/reject workflow.

Problem It Solves

The app can suggest annotations, but suggestions should be directly actionable.

User-Facing Behavior

Replace the text-only suggestion output with a table:

```
Suggested Annotations
-----
```

```
Apply | Tag | Target Field | Reason | Severity | Comment
[ ] | Needs Review | Sensor Brand/Model | Sensors are present but model is
blank | Warning | ...
[ ] | Missing Evidence | Photos Taken? | Audit is Complete but photos are No |
Warning | ...
[ ] | Data Conflict | Cup Type/Material | Mechanical gripper row contains
Silicone | Error | ...
```

Buttons:

```
Apply Selected
Apply All High Confidence
Ignore Selected
Create Follow-Up Actions
Open Target Field
```

How It Works

The backend already has suggested annotations. Expand the suggestion object with:

- Suggestion ID.
- Severity.
- Confidence.
- Target type.
- Audit ID.
- Field key.
- Tag name.
- Reason.
- Suggested comment.
- Whether a follow-up action should be offered.

Implementation Notes

Create:

```
app/widgets/suggested_annotations_table.py
core/annotations/suggestion_workflow.py
```

The UI should:

1. Load suggestions for current audit form.
2. Display them in a table.
3. Allow user to apply selected suggestions.
4. Refresh field tag indicators.
5. Refresh Open Items panel.

6. Log activity.

Expected Final Result

Suggestions become a real review workflow instead of a passive wall of text.

B2. Unified Open Items / Action Board

Goal

Create one central board for unresolved notes, tags, validation findings, missing evidence, and follow-ups.

Problem It Solves

Open work can currently exist in several forms:

- Notes.
- Tags.
- Validation warnings.
- Missing fields.
- Documentation gaps.
- Photo gaps.
- Action items.
- Scheduled summary misses.

These need to appear in one place.

User-Facing Behavior

Add a page:

Open Items

Layout:

Open Items Board

Filters: [All] [Needs Review] [Missing Evidence] [Data Conflict] [Follow-Up Due]
[Documentation Gap] [Validation]

Search: _____

Severity | Type | Machine | Audit ID | Field | Message | Due | Actions

Actions:

- Open target.
- Mark resolved.
- Add note.
- Add/change tag.
- Create action item.
- Export open items report.

Item Sources

The board should merge:

- Annotation tag assignments.
- Open notes.
- Due follow-ups.
- Structured validation findings.
- Photo evidence gaps.
- Scheduled report missed-date findings.
- Action Items sheet entries.

Suggested Data Model

```
@dataclass
class OpenItem:
    id: str
    source: str
    severity: str
    category: str
    title: str
    message: str
    audit_id: str | None
    machine: str | None
    field: str | None
    target_id: str | None
    due_date: str | None
    status: str
    recommended_action: str
    created_at: str
    updated_at: str
```

Implementation Notes

Create:

```
core/open_items.py
app/pages/open_items.py
```

Add Open Items to navigation under Capture or System. It may eventually replace smaller summary panels.

Expected Final Result

The user has one reliable place to see what still needs attention.

B3. Tag-to-Workbook Sync Improvements

Goal

Make workbook color sync useful without creating excessive backups or slowing saves.

Problem It Solves

Tag colors in the workbook are helpful, but writing Excel files repeatedly can be slow and backup-heavy.

User-Facing Behavior

Add settings:

```
Workbook Tag Color Sync
-----
[ ] Sync immediately when tags change
[x] Sync when audit is saved
[x] Batch sync all tags during Deep Refresh
[ ] Disable workbook color sync
```

Add manual button:

```
Sync Annotation Colors to Workbook
```

Show result:

```
Synced 18 tagged audit fields in 1.42 seconds.
Backups created: 1
Warnings: 0
```


Implementation Notes

Prefer batch sync when possible. Keep single-target sync available for small edits.

Add backup retention cleanup so color sync does not produce unlimited backups.

Expected Final Result

Tags remain visible in Excel, but sync behavior is controlled and predictable.

Feature Group C: Validation and Workbook Health

C1. Structured Validation Findings

Goal

Convert validation warnings into structured records that the UI can sort, filter, open, export, and repair.

Problem It Solves

Plain warning strings are readable but hard for the app to act on.

User-Facing Behavior

Workbook Health page should show a table:

Severity	Sheet	Row	Audit ID	Field	Finding	Recommended Action	Fix
----------	-------	-----	----------	-------	---------	--------------------	-----

Example:

Warning	EOAT Inventory	17	AUD-20260527-004	Sensor Brand/Model	Applicable major field is N/A	Fill value or create follow-up	Open Field
---------	----------------	----	------------------	--------------------	-------------------------------	--------------------------------	------------

Suggested Model

```
@dataclass
class ValidationFinding:
    id: str
    severity: str
    category: str
    sheet: str
    row_number: int | None
```

```
audit_id: str | None
machine: str | None
field: str | None
message: str
current_value: str | None
expected_behavior: str | None
recommended_action: str
fix_available: bool
fix_id: str | None = None
```

Finding Categories

- Missing required field.
- Missing applicable major field.
- Non-applicable field correctly N/A.
- Stale hidden value.
- Hybrid completeness warning.
- Semantic consistency warning.
- Invalid dropdown value.
- Invalid numeric value.
- Duplicate Audit ID.
- Legacy header present.
- Missing workbook sheet.
- Missing workbook header.
- Robot Info small workbook issue.
- Schedule/task progress issue.

Implementation Notes

Keep existing markdown reports, but generate both:

```
Foundation_Validation_YYYY-MM-DD_HHMM.md
Foundation_Validation_YYYY-MM-DD_HHMM.json
```

The markdown can summarize the findings. The JSON powers UI navigation and auto-fix.

Expected Final Result

Workbook Health becomes interactive and actionable rather than just a report generator.

C2. Validation Severity Levels

Goal

Make workbook health easier to interpret.

Severity Levels

Use these levels:

```
BLOCKER  
ERROR  
WARNING  
INFO  
AUTO_FIXABLE
```

Recommended meanings:

- BLOCKER: Cannot trust or safely continue until fixed.
- ERROR: Incorrect data or structure, but app may still run.
- WARNING: Needs review, likely incomplete or suspicious.
- INFO: Useful context, not a problem.
- AUTO_FIXABLE: App can safely repair with user approval.

Examples

```
BLOCKER: Master workbook missing.  
ERROR: Duplicate Audit ID.  
WARNING: Applicable major field is N/A.  
INFO: Non-applicable field correctly stored as N/A.  
AUTO_FIXABLE: Non-applicable hidden field contains stale value.
```

Expected Final Result

Health status becomes meaningful. The user can prioritize actual problems instead of reading a blob of warnings like ancient runes.

C3. One-Click Safe Fixes

Goal

Allow the app to safely fix objective workbook problems.

Safe Fix Examples

Safe auto-fixes include:

- Clear non-applicable stale values to N/A.
- Rename legacy `Number of Vacuum Cups` to `Number of Parts Picked` when safe.
- Add missing detail headers.
- Refresh Audit by Press generated view.
- Rebuild dropdown data validation.
- Reapply workbook formatting.
- Refresh compatibility-derived values from source audit.

Unsafe fixes should not be automatic:

- Guessing a sensor model.
- Guessing cup size.
- Guessing whether photos were taken.
- Guessing whether a problem exists.
- Guessing gripper details.

User-Facing Behavior

Workbook Health page:

Auto-Fixable Findings: 9
[Preview Fixes] [Apply Selected Fixes] [Apply All Safe Fixes]

Preview should show:

Row 14, Cup Type/Material: Silicone -> N/A
Reason: Mechanical / Gripper EOAT does not use vacuum tooling fields.

Required Safety Steps

Every auto-fix must:

1. Create a workbook backup.
2. Write an activity log entry.
3. Export a repair report.
4. Re-run validation after fixing.
5. Show before/after counts.

Expected Final Result

Workbook cleanup becomes faster and safer.

C4. Consistency Checks Between Physical and Compatible Rows

Goal

Ensure compatibility rows stay in sync with their physical source audit.

Problem It Solves

When a physical audit changes, compatible rows may become stale if inherited values are not refreshed correctly.

User-Facing Behavior

Add a Compatibility Health section:

```
Compatibility Coverage
-----
Source Audit: AUD-20260527-003
Compatible Rows: 14
Rows in Sync: 13
Rows Stale: 1
Rows Missing: 0
Rows Extra: 2
```

Actions:

```
Regenerate Compatibility Rows
Review Stale Rows
Open Source Audit
Export Compatibility Health Report
```

Implementation Notes

Create:

```
core/compatibility_health.py
```

Compare source audit fields against compatible rows for inherited/system-managed fields.

Expected Final Result

Compatibility entries remain trustworthy after edits.

C5. Compatibility Impact Preview

Goal

Show what will happen to compatibility rows before saving an edited physical audit.

User-Facing Behavior

Before saving an edited audit with existing compatible entries:

```
Compatibility Impact Preview
```

```
-----
```

```
This audit has 14 compatible entries.
```

```
The following inherited fields may update:
```

- EOAT Type
- EOAT Moves
- Connection Type
- Tool #
- Part Family

```
[Save and Regenerate Compatibility] [Save Audit Only] [Cancel]
```

Implementation Notes

The preview should not modify files. It should calculate affected rows and fields.

Expected Final Result

The user understands save impact before workbook rows change.

Feature Group D: Dashboard, Navigation, and Speed

D1. Page Registry

Goal

Replace hardcoded dashboard page mapping and if/elif page construction with a registry.

Problem It Solves

The dashboard has many pages. Adding more pages with hardcoded navigation and import branches will become difficult to maintain.

Suggested Model

```
@dataclass(frozen=True)
class PageSpec:
    key: str
    label: str
    section: str
    factory_path: str
    requires_config: bool = True
    refresh_on_show: bool = False
    description: str = ""
```

Example registry:

```
PAGES = [
    PageSpec("home", "Home", "Overview", "app.pages.home:HomePage"),
    PageSpec("audit", "EOAT Audit", "Capture",
"app.pages.audit.page:AuditPage"),
    PageSpec("open_items", "Open Items", "Capture",
"app.pages.open_items:OpenItemsPage"),
]
```

Expected Final Result

Navigation, page creation, command palette search, and page lifecycle management become cleaner.

D2. Page Lifecycle Hooks

Goal

Give pages standard hooks so the app can refresh and close safely.

Suggested Interface

```
class DashboardPageProtocol:
    def on_show(self) -> None: ...
    def on_hide(self) -> None: ...
    def on_project_root_changed(self, config) -> None: ...
```

```
def on_event(self, event) -> None: ...
def can_close(self) -> tuple[bool, str]: ...
```

Example Uses

- Audit page warns about unsaved changes.
- Home page quick-refreshes on show.
- Notes page refreshes after annotation changes.
- Workbook Health refreshes after validation.
- Scheduled Reports checks status when opened.

Expected Final Result

Pages update predictably without scattered hardcoded refresh calls.

D3. App-Level Event Bus

Goal

Allow pages to react to important project changes without directly knowing about each other.

Events

Add events such as:

```
AuditSaved
AuditLoaded
AnnotationChanged
TagColorSynced
RobotInfoUpdated
CompatibilityRegenerated
WorkbookValidated
ReportGenerated
ProjectRootChanged
DashboardCacheUpdated
ScheduledReportRan
```

Suggested Model

```
@dataclass
class AppEvent:
    event_type: str
```



```
payload: dict[str, object]
timestamp: str
```

Expected Final Result

The app becomes easier to extend because pages can refresh through events instead of tight coupling.

D4. Dashboard Cache Hardening

Goal

Make quick dashboard load reliable and accurate.

Problem It Solves

The dashboard cache should go stale when important source files change. New annotation and Robot Info data should participate in cache invalidation.

Add These Sources

Add to cache metadata:

- Annotation SQLite database.
- Robot_Info.xlsx.
- Photo index files.
- Notes/tag export files.
- Open items outputs.
- Recent validation JSON files.
- Scheduled reports log.

User-Facing Behavior

Instead of only saying “stale,” show why:

```
Dashboard cache is stale because:
- Robot_Info.xlsx changed 3 minutes ago.
- annotations.sqlite changed 1 minute ago.
- scheduled_tools.log changed today.
```

Expected Final Result

The user can trust cached dashboard data and understand when deep refresh is needed.

D5. Structured Performance Logging and Performance Page

Goal

Track slow operations clearly and make performance improvements measurable.

Current Need

The app has performance logging. Convert or supplement it with JSONL to enable dashboards.

Suggested Log Format

```
{
  "timestamp": "2026-05-27T18:42:00",
  "operation": "dashboard.deep_refresh",
  "duration_seconds": 4.31,
  "success": true,
  "details": {},
  "project_root_mode": "real"
}
```

Performance Page

Add page:

```
Performance
-----
Slowest recent operations:
1. dashboard.deep_refresh - 8.4s
2. audit_save_entry - 5.2s
3. annotation_color_sync - 2.1s

Startup:
Shell visible: 1.3s
Config load: 0.1s
Window create: 0.8s
```

Tests

Add performance smoke tests with generous thresholds for demo data:

- App shell visible within reasonable time.
- Quick refresh does not do deep workbook analysis.
- Deep refresh runs in background.

Expected Final Result

Performance can be improved scientifically instead of by vibes and caffeine.

D6. Global Command Palette

Goal

Add fast keyboard-driven navigation and actions.

User-Facing Behavior

Keyboard shortcut:

```
Ctrl + K
```

Command examples:

```
Open audit AUD-20260527-004
Go to machine 12
Run workbook validation
Open latest weekly report
Deep refresh dashboard
Open Robot Info workbook
Show Needs Review tags
Generate daily summary
```

Implementation Notes

Commands should come from registries:

- Page registry.
- Tool registry.
- Recent audits.
- Recent reports.
- Open items.
- Tags.

Create:

```
app/widgets/command_palette.py
core/commands.py
```

Expected Final Result

Power users can move through the app much faster.

D7. Global Search

Goal

Search across audits, notes, tags, fields, reports, machines, and small Robot Info workbook values.

User-Facing Behavior

Add a global search bar on Home or in the command palette.

Results grouped by type:

Audits

- AUD-20260527-004 | Machine 12 | Hybrid

Fields

- Sensor Brand/Model | Needs Review | AUD-20260527-004

Notes

- Check reed switch mounting

Tags

- Missing Evidence | 8 targets

Reports

- Week2_Day3_Status_2026-05-27.md

Search Operators

Support simple filters later:

```
tag:"Needs Review"
machine:12
audit:AUD-20260527
status:open
due:<2026-06-01
```

Expected Final Result

The app becomes easier to use as the project grows.

Feature Group E: Scheduled Reports and Automation

E1. Scheduled Report Calendar Preview

Goal

Make scheduled daily and weekly summary automation visible and trustworthy.

User-Facing Behavior

Scheduled Reports page should show a 14-day preview:

Date	Expected Job	Status
Mon May 25	Daily Summary	Already exists
Tue May 26	Daily Summary	Missed
Wed May 27	Daily Summary	Due tonight at 7:00 PM
Thu May 28	Daily Summary	Pending
Fri May 29	Weekly Summary	Pending

Actions

- Run due summaries now.
- Generate missed selected summaries.
- Mark selected as intentionally skipped.
- Open report folder.
- Open scheduled log.

Implementation Notes

Use existing scheduler decision logic. Add a preview function that does not run reports.

Suggested function:

```
def preview_schedule(project_root, start_date, days=14) ->
    list[ScheduledPreviewRow]
```

Expected Final Result

The user can see exactly what will run and what already happened.

E2. Catch-Up Summaries

Goal

Allow missed daily/weekly summaries to be generated after the fact.

User-Facing Behavior

```
Missed Summaries
-----
[ ] 2026-05-25 Daily
[ ] 2026-05-26 Daily
[ ] 2026-05-22 Weekly

[Generate Selected Catch-Up Reports]
[Mark Selected Intentionally Skipped]
```

Catch-Up Behavior

Catch-up summaries should:

- Use the correct report date.
- Use the correct resolved project week/day.
- Save in the correct report folder.
- Be clearly marked as catch-up if desired.
- Not overwrite existing reports.

Expected Final Result

Missed automation does not permanently break the reporting trail.

E3. Smarter Scheduled Report Content

Goal

Generate scheduled summaries from actual project activity instead of generic filler.

Data Sources

Daily summaries should pull from:

- Activity log.
- Audit saves.
- New notes/tags.
- Open items created/resolved.
- Validation runs.
- Reports generated.
- Schedule task progress.
- Files modified.
- Blockers.

Weekly summaries should pull from:

- Daily summaries.
- Weekly activity logs.
- Audit progress delta.
- Open item changes.
- Pilot candidate changes.
- FMEA/KPI changes.
- New deliverables.

Example Output

Completed Today

- Saved audit AUD-20260527-004 for Machine 12.
- Updated Robot_Info.xlsx robot-side pneumatic circuit counts.
- Applied 3 Needs Review tags.
- Generated validation report.

Open Items

- Sensor Brand/Model missing for AUD-20260527-004.
- Quick disconnect photo evidence missing.

Next Recommended Work

- Finish photo evidence for Machine 12.
- Review validation warnings before marking audit Complete.

Implementation Notes

Create:

```
core/report_context.py
```

This module should build a structured report context object before markdown generation.

Expected Final Result

Automatic reports become actually useful.

E4. Windows Task Scheduler Preflight Diagnostics

Goal

Make scheduled automation easier to diagnose.

User-Facing Behavior

Scheduled Reports page should show:

```
Scheduled Reports Preflight
-----
PowerShell available: Yes
Task Scheduler accessible: Yes
Daily task installed: Yes
Weekly task installed: No
Project root valid: Yes
Python executable found: Yes
Run scripts found: Yes
Can write to reports folder: Yes
Last daily run: Success
Last weekly run: Not installed
```

Buttons:

```
Run Dry-Run Daily
Run Dry-Run Weekly
Install/Repair Tasks
Uninstall Tasks
Open Task Scheduler
Copy Diagnostic Summary
```

Implementation Notes

Use subprocess checks safely with timeouts. Do not block UI.

Expected Final Result

The user can tell whether scheduled automation is installed, healthy, and capable of writing reports.

Feature Group F: Photos and Evidence

F1. Photo Evidence Coverage

Goal

Track whether each audit has the photos needed to support the recorded data.

Problem It Solves

A single `Photos Taken? = Yes` field is not enough. Different types of photos support different audit claims.

Evidence Categories

Suggested categories:

- Overall EOAT.
- Robot connection / mounting.
- EOAT-side pneumatic circuits.
- Robot-side pneumatic circuits.
- Vacuum cups or grippers.
- Sensors.
- Quick disconnects.
- Tubing routing.
- Cable management.
- Mounting hardware.
- Wear/damage.
- Nameplates/labels if relevant.

User-Facing Behavior

For each audit:

```
Photo Evidence Coverage
-----
Overall EOAT: Yes
Sensors: Missing
Quick Disconnects: Yes
```

Tubing Routing: Missing
Mounting Hardware: Yes

Validation should warn if:

- Audit is marked Complete but required photo categories are missing.
- Known Issues mentions wear/damage but no wear/damage photo exists.
- Sensors Present? is Yes but no sensor photo exists.
- Quick Disconnects Present? is Yes but no quick disconnect photo exists.

Implementation Notes

Create:

core/photo_evidence.py

Suggested model:

```
@dataclass
class PhotoEvidenceCategory:
    key: str
    label: str
    required_when: Callable[[dict], bool]

@dataclass
class PhotoEvidenceStatus:
    audit_id: str
    category: str
    present: bool
    photo_count: int
    required: bool
    warning: str | None
```

Expected Final Result

Photos become evidence instead of a vague yes/no checkbox.

F2. Phone-Friendly Photo Intake

Goal

Make it easier to collect and organize photos taken on a phone.

User-Facing Behavior

Add a Photo Intake Session workflow:

```
Photo Intake Session
-----
Audit ID: AUD-20260527-004
Machine: 12
Expected photos:
1. Overall EOAT
2. Sensors
3. Quick disconnects
4. Tubing routing

[Create Intake Folder]
[Copy Folder Path]
[Export Photo Checklist]
```

Optional future enhancement:

- Generate a QR code for a local upload page.
- Run a local-only temporary web uploader.

Minimum Version

Start simple:

- Create an audit-specific intake folder.
- Copy folder path.
- Export checklist markdown.
- Let user drag/import photos later.

Expected Final Result

Photo organization becomes easier and less dependent on remembering what `IMG_4829.jpg` was supposed to show.

Feature Group G: Analysis, FMEA, Standards, and Pilot Selection

G1. Pilot Candidate Evidence Packets

Goal

Generate a structured evidence package for each pilot candidate.

User-Facing Behavior

Pilot Candidate page should include:

Generate Evidence Packet

Output should include:

- Machine.
- Audit ID.
- EOAT Type.
- Tool number.
- Part family/name.
- Known issues.
- Drop/mis-pick history.
- Cycle time concern.
- Scrap/quality concern.
- Maintenance frequency.
- Tags.
- Notes.
- Photo evidence.
- FMEA risks.
- Expected improvement.
- Implementation difficulty.
- Recommended action.

Output Formats

- Markdown.
- Optional DOCX later.
- Optional presentation-ready summary.

Expected Final Result

Pilot choices are backed by evidence rather than memory.

G2. FMEA Generation from Tags and Issues

Goal

Use audit data, tags, notes, and validation findings to suggest FMEA-lite entries.

User-Facing Behavior

FMEA page should show suggested rows:

```
Suggested Failure Mode: Sensor failure / false part-present
Evidence:
- Sensors Present? = Yes
- Sensor Brand/Model missing
- Tag: Needs Review
Suggested Severity: 7
Suggested Occurrence: 4
Suggested Detection: 5
Suggested Mitigation: Verify sensor model and inspect wiring.

[Accept] [Edit] [Reject]
```

Inputs

- Known Issues.
- Drop/Mis-Pick History.
- Maintenance Frequency.
- Notes.
- Tags.
- Missing evidence.
- Validation findings.
- Semantic warnings.

Important Rule

The app may suggest FMEA values, but the user must confirm or edit them. The app should not silently decide engineering risk numbers.

Expected Final Result

FMEA creation becomes faster and better connected to actual audit evidence.

G3. Standards Compliance Score

Goal

Score each EOAT against the standardization goals.

Example Categories

- EOAT classification complete.
- Tooling details complete.
- Sensor standard documented.
- Quick disconnect standard documented.
- Tubing/routing condition acceptable.
- Cable management acceptable when wiring applies.
- Mechanical condition acceptable.
- Photo evidence complete.
- Documentation status complete.
- Spare parts identified.
- Known issues reviewed.

User-Facing Behavior

```
Standards Compliance
-----
Quick disconnect standard: Pass
Sensor documentation: Warning
Tubing routing: Warning
Documentation/BOM: Fail
Photo evidence: Pass
Spare parts: Fail

Overall Score: 61%
```

Implementation Notes

Create:

```
core/standards_compliance.py
```

The scoring should be transparent. Each point should have a reason.

Expected Final Result

The project can show measurable standardization progress.

Feature Group H: Audit Organization and Press View

H1. Press View / Machine View

Goal

Organize audits by press/machine inside the app.

Problem It Solves

Some presses may have many tools. A flat audit list becomes difficult to navigate.

User-Facing Behavior

Add page:

Press View

Example:

```
Press 1
- Physical audits: 2
- Compatible entries: 14
- Open items: 6
- Photo gaps: 3
- Status: In Progress

Tools:
Tool # | Audit ID | Entry Type | EOAT Type | Status | Open Items | Photos | Last
Updated
```

Actions:

- Open audit.
- Duplicate audit for another tool.
- Generate compatible entries.
- View open items for press.
- Export press summary.

Implementation Notes

Create:

```
core/press_view.py
app/pages/press_view.py
```

This should not replace the existing Audit by Press workbook sheet. It should make press organization usable inside the app.

Expected Final Result

The user can handle presses with many tools without losing track.

H2. Audit Revision History

Goal

Track what changed when audits are saved.

User-Facing Behavior

Audit page should have a History panel:

```
Audit History
-----
2026-05-27 18:41 | Sensor Brand/Model changed N/A -> SMC
2026-05-27 18:42 | Robot_Info.xlsx updated
2026-05-27 18:42 | Compatibility entries regenerated
```

Data Model

```
{
  "audit_id": "AUD-20260527-004",
  "timestamp": "2026-05-27T18:41:00",
  "event": "audit_saved",
  "changed_fields": {
    "Sensor Brand/Model": ["N/A", "SMC"],
    "Part-Present Detection Present?": ["No", "Yes"]
  },
  "files_modified": [],
  "warnings": []
}
```


Implementation Notes

Store history in local ignored project data:

```
00_Project_Admin/history/audit_history.jsonl
```

Expected Final Result

The user can trace workbook changes and recover from mistakes.

Feature Group I: File Safety, Backups, Locks, and Release Readiness

I1. Workbook Lock Detector

Goal

Detect when Excel workbooks are locked before attempting write operations.

User-Facing Behavior

Show workbook status:

```
Workbook Status
-----
EOAT_Master_Tracker.xlsx: Available
Robot_Info.xlsx: Locked by Excel
Pending syncs: 1
```

If save fails due to lock:

```
The master workbook appears to be open or locked.
No changes were written.

Options:
[Retry] [Save Draft] [Open Workbook Folder]
```

Implementation Notes

Create:

```
core/workbook_locks.py
```

Detect lock by attempting safe temporary open/write checks where appropriate. Avoid destructive checks.

Expected Final Result

Workbook write failures become understandable and recoverable.

I2. Backup Retention Cleanup

Goal

Prevent workbook backup folders from growing forever.

Policy

Suggested default:

- Keep all backups from the last 7 days.
- Keep the newest 25 backups per workbook.
- Keep backups associated with failed validations.
- Allow manual cleanup preview before deletion.

User-Facing Behavior

Backup Manager page or Settings section:

```
Backup Manager
-----
Master Tracker backups: 42
Robot_Info backups: 17
Total size: 184 MB
Safe cleanup candidates: 21

[Preview Cleanup] [Clean Safe Old Backups]
```

Expected Final Result

Backups remain useful without turning the project folder into sedimentary rock.

I3. Release Readiness Page

Goal

Make safe GitHub publishing easier.

User-Facing Behavior

Add page:

```
Release Readiness
-----
pytest: Not run
repo safety audit: Passed
git status: Dirty
local config staged: No
real workbooks staged: No
private paths found: No
demo data valid: Yes
README/USAGE updated: Yes
```

Buttons:

```
Run Tests
Run Repo Safety Audit
Show Staged Files
Open Sanitization Report
Copy Commit Checklist
```

Expected Final Result

The user can confidently commit/push without accidentally exposing private data.

I4. Staged-File NDA Scanner

Goal

Detect unsafe files or content before committing.

What to Scan For

Blockers:

- Real project roots.
- Real workbooks outside demo/template folders.
- Real reports/logs/cache files.
- Local config files.
- Photos.
- Capacity files.
- Internal shared-drive paths.
- Mold numbers, part numbers, customer names, or other sensitive internal identifiers.

Warnings:

- Public company references combined with operational context.
- Suspicious paths.
- Large binary files.

User-Facing Behavior

```
Repo Safety Audit
-----
BLOCKER: Real workbook staged: ...
WARNING: Internal path-like string found in docs/example.md
INFO: Demo workbook allowed
```

Implementation Notes

Extend existing safety audit scripts rather than creating a totally separate system unless necessary.

Expected Final Result

Publishing is safer and less dependent on memory.

Feature Group J: Config and Settings

J1. Config-Driven Defaults

Goal

Move audit defaults out of hardcoded UI logic into user-editable config/reference data.

Defaults to Move

- Default auditor.
- Default plant/area.
- Default cleanroom state.
- Default quick disconnect presence.
- Default pneumatic quick disconnect type.
- Default vacuum generator type.
- Default documentation/photo values.
- Default reliability unknown values.
- Connection type to changeover difficulty mapping.
- Gripper presets.

Suggested Config

```
{
  "audit_defaults": {
    "auditor": "Kato Gray",
    "plant_area": "Plant 4",
    "cleanroom": "Whiteroom",
    "status": "In Progress",
    "priority": "Medium",
    "follow_up_needed": "No",
    "quick_disconnects_present": "Yes",
    "pneumatic_quick_disconnect_type": "PTC",
    "vacuum_generator_type": "Venturi",
    "eoat_interchangeable_circuits": "0",
    "robot_interchangeable_circuits": "0"
  },
  "connection_defaults": {
    "ATI": {
      "changeover_difficulty": "Easy"
    },
    "DoveTail": {
      "changeover_difficulty": "Medium"
    }
  }
}
```

Important Note

If current code uses `Low` for ATI but the desired user-facing default is `Easy`, the defaults should be updated consistently across UI, normalization, validation, tests, and documentation.

Expected Final Result

Defaults can be changed without editing Python.

J2. Expanded Settings Page

Goal

Make operational configuration visible and editable.

Settings Sections

1. Project:
2. Project root.
3. Demo mode status.
4. Project start date.
5. Workdays.
6. Holidays.
7. Audit Defaults:
8. Auditor.
9. Plant/Area.
10. Cleanroom default.
11. Default status/priority.
12. Quick disconnect defaults.
13. Sensor defaults.
14. Scheduled Reports:
15. Daily summary days.
16. Daily summary time.
17. Weekly summary day.
18. Weekly summary time.
19. Timezone.
20. Standards:
21. Connection defaults.
22. Gripper presets.
23. Documentation requirements.

24. Safety:

25. NDA safety audit enabled.

26. Backup retention.

27. Confirm before workbook repairs.

Expected Final Result

The app becomes adaptable without code changes.

J3. Gripper Presets as Configurable Reference Data

Goal

Turn gripper presets into a small configurable parts reference.

Current Concept

Friendly gripper names map to workbook part/model numbers.

Expanded Data

```
{
  "friendly_name": "Large Double Gripper",
  "part_number": "MHZL2-16D",
  "manufacturer": "SMC",
  "default_type": "Double Pressure",
  "notes": ""
}
```

User-Facing Behavior

- Gripper Model dropdown shows friendly names.
- Workbook saves part/model number.
- UI can display both friendly name and saved model number.
- Settings can add/edit presets later.

Expected Final Result

Gripper data is easier to maintain.

Optional Architecture Enhancement: Local SQLite Audit Mirror

Goal

Improve performance, search, history, and resilience by mirroring audit state into local SQLite.

Important Constraint

Excel remains the required final deliverable. SQLite should support the app, not replace the workbook deliverable.

Benefits

- Faster dashboard queries.
- Better global search.
- Easier revision history.
- Pending sync queue if workbook is locked.
- Structured relationships between audits, fields, notes, tags, photos, and findings.

Suggested Tables

```
audits
audit_fields
compatibility_entries
robot_circuit_counts
photo_evidence
validation_findings
open_items
audit_history
pending_workbook_syncs
```

Implementation Approach

Do not add this first. Add after the app has structured validation findings and an event bus.

Initial version can be read-through cache only:

1. Read workbook.
2. Populate SQLite mirror.
3. Use SQLite for search/dashboard.
4. Keep workbook as source of truth.

Later version can support pending write syncs.

Expected Final Result

The app becomes faster and more reliable without abandoning Excel outputs.

SQLite Full-Text Search for Notes and Tags

Goal

Improve search quality for notes and tags.

Problem It Solves

LIKE-based search works early, but FTS is better as notes grow.

Implementation

Add SQLite FTS5 table for notes:

```
CREATE VIRTUAL TABLE notes_fts USING fts5(  
    subject,  
    body_markdown,  
    collection,  
    note_type  
);
```

Add triggers or service-level updates when notes are created/updated/archived.

Expected Final Result

Search feels faster and more useful as the project grows.

Minimal Robot Info Scope

Purpose

Robot_Info.xlsx exists only to store robot-side pneumatic circuit counts that do not belong in the EOAT Master Tracker.

Included Fields

Keep the workbook focused on:

- Plant/Area.
- Machine Number.
- Robot Vacuum Circuits.
- Robot Pressure Circuits.
- Robot Interchangeable Circuits.
- Last Audit ID.
- Last Updated.
- Notes.

Excluded Fields

Do not expand Robot_Info.xlsx into a full robot master database. Do not duplicate fields that already exist in the Master Press List, such as:

- Robot model/controller.
- Robot serial number.
- Robot manufacturing date.
- Press details.
- Full robot identity metadata.
- Deep machine reference details.

App Behavior

The app may use the Master Press List for robot reference lookup. It may use Robot_Info.xlsx only for the three robot-side circuit values.

Expected Final Result

Robot-side circuit data is separated cleanly without duplicating the Master Press List.

Implementation Roadmap

Phase 1: Architecture Foundation

Goals

Prepare the app for growth without breaking current behavior.

Tasks

1. Add page registry.
2. Add page lifecycle hooks.
3. Add event bus.
4. Add dirty-form foundation.
5. Add structured validation finding model.
6. Add config-driven defaults foundation.
7. Update tests to ensure existing behavior still works.

Acceptance Criteria

- App opens with same visible pages.
 - Navigation still works.
 - Existing tests pass.
 - Audit save/load still works.
 - Scheduled Reports page still works.
 - No real data added to repo.
-

Phase 2: Audit Guidance

Goals

Make the audit page actively guide the user.

Tasks

1. Add Audit Coach panel.
2. Add section completion calculation.
3. Add hidden-field reason display.
4. Add Finish This Audit workflow.
5. Add direct navigation to missing fields.
6. Add create follow-up / tag Needs Review actions.

Acceptance Criteria

- Audit Coach updates when EOAT Type changes.
 - Coach respects field applicability rules.
 - Hidden fields show correct reasons.
 - Finish This Audit only shows applicable missing fields.
 - Hybrid and semantic warnings appear in coach.
-

Phase 3: Open Items and Suggestions

Goals

Unify notes, tags, validation warnings, and follow-ups.

Tasks

1. Add Suggested Annotations table.
2. Add Apply/Ignore workflow.
3. Add Open Items page.
4. Merge open notes, tags, validation findings, and photo gaps.
5. Add Open Target action.
6. Add Mark Resolved action.

Acceptance Criteria

- Suggestions can be applied without manual tag dialog.
 - Open Items shows unresolved annotation items.
 - Clicking an audit-field item opens the correct audit field.
 - Counts refresh after applying/resolving items.
-

Phase 4: Scheduled Reports and Automation Trust

Goals

Make automation visible, diagnosable, and useful.

Tasks

1. Add scheduled report calendar preview.
2. Add catch-up summary workflow.
3. Add Task Scheduler preflight diagnostics.
4. Improve report context builder.
5. Generate scheduled report content from real activity/open items.

Acceptance Criteria

- Scheduled Reports page shows next expected runs.
 - Missed summaries can be generated manually.
 - Dry-run daily/weekly tests work.
 - Reports include actual recent project activity when available.
-

Phase 5: Validation Repair and Workbook Safety

Goals

Make workbook health actionable and safe.

Tasks

1. Export validation JSON.
2. Add Workbook Health findings table.
3. Add severity levels.
4. Add safe auto-fix preview/apply.
5. Add workbook lock detector.
6. Add backup retention manager.

Acceptance Criteria

- Findings table can filter by severity/category.
 - Findings can open target audit fields.
 - Safe fixes preview changes before applying.
 - Auto-fixes create backups and activity logs.
 - Locked workbook errors are user-friendly.
-

Phase 6: Search, Command Palette, and Press View

Goals

Make navigation fast and project-scale organization easier.

Tasks

1. Add command palette.
2. Add global search.
3. Add Press View page.
4. Add audit revision history panel.
5. Add optional SQLite FTS for notes/tags.

Acceptance Criteria

- Ctrl+K opens command palette.
 - User can search audits, notes, tags, reports, and machines.
 - Press View groups physical and compatible entries.
 - Audit history records field changes.
-

Phase 7: Evidence, Analysis, and Handoff Strength

Goals

Make final deliverables stronger and easier to justify.

Tasks

1. Add photo evidence coverage.
2. Add phone-friendly photo intake session.
3. Add standards compliance score.
4. Add FMEA suggestions from tags/issues.
5. Add pilot candidate evidence packets.
6. Add release readiness page.
7. Add staged-file NDA scanner enhancements.

Acceptance Criteria

- Audit marked Complete warns if required photo evidence is missing.
 - Pilot packet exports markdown evidence summary.
 - FMEA suggestions require user confirmation.
 - Release readiness page detects unsafe staged files.
-

Testing Strategy

Unit Tests

Add tests for:

- Field applicability and hidden reasons.
- Audit Coach summary generation.
- Completion calculation.
- ValidationFinding generation.
- Safe auto-fix preview.
- Config default loading.
- Scheduled report preview.
- Catch-up summary date/week/day resolution.
- Open item aggregation.
- Photo evidence requirements.
- Press View grouping.
- Dirty-form diff detection.

UI Tests

Add tests for:

- Audit Coach appears and updates.
- Finish This Audit navigates fields.
- Suggested annotations can be applied.
- Open Items opens target audit field.
- Dirty-form warning appears before destructive actions.
- Scheduled Reports preflight displays status.
- Command palette opens and navigates.

Integration Tests

Add tests for:

- Save audit -> compatibility autorun -> Robot_Info small workbook update -> annotation sync -> event bus refresh.
- Validation -> findings JSON -> Workbook Health table.
- Apply safe fixes -> backup -> workbook modified -> validation re-run.
- Generate catch-up summary -> no duplicate overwrite.

Safety Tests

Add tests for:

- Real data patterns blocked by safety audit.
- Local config files ignored.
- Demo data still usable.
- Cache files not committed.
- Generated reports/logs excluded from repo.

Codex Execution Prompt

Use this prompt for Codex when starting the next pass:

```
Review the EOAT Command Center repository on branch fix/scheduled-reports-  
performance at commit 38b9e6f.
```

```
I want to implement the next major feature expansion described in the canvas  
document named "EOAT Command Center Feature Expansion Specification."
```

```
Important scope clarification:
```

- ```
- Do NOT implement the previously proposed full Robot Info entity system.
```

- Robot\_Info.xlsx should remain a small workbook only for Robot Vacuum Circuits, Robot Pressure Circuits, and Robot Interchangeable Circuits, plus basic tracking fields like Plant/Area, Machine Number, Last Audit ID, Last Updated, and Notes.
- Deeper robot details already live in the Master Press List and should not be duplicated into Robot\_Info.xlsx.

Primary goal:

Turn the app into a guided engineering workflow system centered around Audit Coach + Open Items.

Implementation order:

1. Architecture foundation: page registry, lifecycle hooks, event bus, dirty-form foundation, config-driven defaults foundation.
2. Audit Coach: section completion, hidden-field reasons, next best action, Finish This Audit workflow.
3. Open Items and Suggestions: suggested annotations table, apply/ignore workflow, unified Open Items page.
4. Scheduled Reports: calendar preview, catch-up summaries, Task Scheduler preflight, smarter report context.
5. Validation and Safety: structured ValidationFinding model, severity levels, findings JSON, safe auto-fix preview/apply, workbook lock detection, backup retention.
6. Navigation and Organization: command palette, global search, Press View, audit revision history.
7. Evidence and Deliverables: photo evidence coverage, photo intake session, standards compliance score, FMEA suggestions, pilot candidate evidence packets, release readiness, staged-file NDA scanner improvements.

Hard requirements:

- Keep the app local-first.
- Keep all real project data outside the repo.
- Do not commit real workbooks, reports, logs, cache files, photos, capacity data, customer names, part numbers, mold numbers, or internal paths.
- Preserve existing CLI tools.
- Preserve demo project behavior.
- Add or update tests for every behavior change.
- Run pytest and repo safety audit before declaring complete.
- Prefer small, reviewable commits.
- Do not bury business logic inside giant UI page classes.



# Expected Final App Experience

When all phases are complete, the app should behave like this:

1. User opens EOAT Command Center.
2. Home loads quickly from cache.
3. App clearly shows real/demo project mode.
4. Open Items shows the most important unresolved work.
5. Audit Coach guides the user through missing applicable fields.
6. Hidden fields are explained instead of mysterious.
7. Suggestions can be applied directly as tags/follow-ups.
8. Workbook Health shows structured findings and safe fixes.
9. Scheduled Reports shows next runs, missed summaries, and diagnostics.
10. Press View organizes audits by machine/press.
11. Global search and command palette make navigation fast.
12. Photo evidence coverage makes audits defensible.
13. Pilot/FMEA/standards outputs are generated from actual evidence.
14. Release Readiness helps prevent unsafe GitHub commits.

The app should feel faster, clearer, safer, and much more helpful during real audit work.

It should not just store information. It should actively guide the project toward a complete, defensible final handoff.